

Практикум по основам Git в Windows

Дисциплина «Управление конфигурацией и сопровождение программного обеспечения». Работа рассчитана на начинающих пользователей Windows 10/11. Сначала преподаватель объясняет действия по слайдам, затем вы выполняете их у себя по этому практикуму.

Что получится

Локальный репозиторий с README.txt и PLAN.txt, тремя коммитами, ветками main и feature-plan и меткой v0.1.0. В конце вы подготовите ZIP с историей и коротким отчётом.

Порядок работы

Первая пара: объяснение 35 минут, самостоятельная работа 55 минут. Между парами перерыв 10 минут. Вторая пара: объяснение 25 минут, самостоятельная работа 60 минут, итоги 5 минут. Итого 180 учебных минут и отдельный перерыв.

Во время объяснения не требуется одновременно вводить команды. Практикум содержит все шаги. Если преподаватель объясняет быстрее, оставшееся время можно использовать для практики.

Сдача позже

Сейчас сохраняйте результаты на своём компьютере. Преподаватель позднее сообщит ссылку и срок загрузки в электронную информационно-образовательную среду, далее ЭИОС. Работа не требует уже созданного курса. Незавершённые шаги можно закончить до объявленного срока.

Что скачать заранее

Этот PDF и 04_Стартовый_проект_Windows.zip. Установщик Git for Windows скачивается отдельно с <https://git-scm.com/install/windows>. Стартовый ZIP не содержит установщик. Скачивание и установка требуют доступа к сети, сами упражнения после подготовки выполняются локально.

Проверка. Все команды ниже вводятся в Git Bash, по одной строке, без символа \$. В Блокноте редактируем только содержимое учебных файлов.

A0 Установка Git и распаковка файлов

Если Git уже установлен

Откройте Git Bash через «Пуск» и выполните `git --version`. Если виден номер версии, повторная установка не требуется. Команды практикума предполагают Git 2.28 или новее. При неизвестном параметре `-b` у `git init` установите актуальный Git for Windows.

Если Git ещё нет

1. В Windows откройте «Параметры», «Система», «О системе» и посмотрите тип системы. Для большинства ПК Intel/AMD нужен вариант x64. Для системы ARM выбирайте ARM64. Если указана 32-разрядная система, сообщите преподавателю: текущий маршрут рассчитан на x64/ARM64.
2. Откройте <https://git-scm.com/install/windows> и скачайте подходящий Setup. Запустите файл из «Загрузок». Проходите страницы установщика с предложенными настройками: Next, Install, Finish. Если установка ограничена политиками компьютера, обратитесь к ответственному сотруднику.
3. Найдите Git Bash через меню «Пуск». Не требуется создавать учётную запись на хостинге репозитория. Если официальный файл не скачивается или программа не запускается, сообщите преподавателю точный текст ошибки.

Распаковка стартового ZIP

4. В Проводнике нажмите правой кнопкой по `04_Стартовый_проект_Windows.zip` и выберите «Извлечь всё». Распакуйте в обычную папку для учебных работ, например в «Документы». Откройте извлечённую папку `git-start`.
5. В `git-start` найдите `00_CHECK_GIT.cmd`, `01_OPEN_GIT_BASH.cmd`, `report_template.txt` и папку `git-practice`. Внутри `git-practice` должен быть `README.txt`. Выполняйте работу в этой папке, а не внутри открытого ZIP.

Проверка. На этом шаге достаточно установленного Git и распакованных файлов. Репозиторий пока не создаём.

Официальный источник установки: <https://git-scm.com/install/windows>

A1 Открытие Git Bash в нужной папке

1. В папке git-start дважды щёлкните по 00_CHECK_GIT.cmd. Это проверка наличия Git в обычных местах установки. Она не устанавливает программы. Прочитайте результат и закройте окно проверки.
2. Запустите 01_OPEN_GIT_BASH.cmd. Помощник попытается открыть Git Bash в папке git-practice. Все дальнейшие команды выполняйте в открывшемся Git Bash, а не в окне проверки.
3. Введите по очереди:

```
git --version  
pwd  
ls
```

Проверка. Виден номер Git. Путь от pwd заканчивается на /git-practice. Команда ls показывает README.txt. Путь и имя пользователя будут своими.

Если помощник не открыл нужное окно

Откройте git-practice в Проводнике, нажмите правой кнопкой по пустому месту и выберите Open Git Bash here. В Windows 11 сначала может потребоваться «Показать дополнительные параметры».

Если такого пункта нет, откройте Git Bash через «Пуск». Наберите cd и пробел, перетащите папку git-practice из Проводника в Git Bash, проверьте кавычки вокруг пути с пробелами и нажмите Enter. Либо используйте cd с полным путём в прямых кавычках. Затем снова выполните pwd и ls.

Правила ввода

Одна команда — Enter — чтение ответа. Для вставки используйте Shift+Insert или Paste в контекстном меню. Знак \$ из приглашения Git Bash и номера шагов не вводите. При незакрытой кавычке и приглашении > нажмите Ctrl+C и повторите строку.

Проверка. Не выполняйте git init в папке с личными документами или во внешней git-start. Нужна именно git-practice.

A2 Репозиторий и автор

Продолжайте только после проверки папки на шаге A1. Для первого выполнения используйте свежую распакованную папку без существующей истории.

Создание репозитория

```
git init -b main
git status
```

Проверка. Ожидаем On branch main и No commits yet. README.txt указан в Untracked files. Папку .git не редактируем и не удаляем.

Настройка автора после git init

Замените Ivan Ivanov своим именем и фамилией. Для локального учебного адреса можно оставить student@example.invalid. Пароль и доступ к почте не нужны.

```
git config user.name "Ivan Ivanov"
git config user.email "student@example.invalid"
git config user.name
git config user.email
```

Проверка. Последние две строки показывают заданные имя и адрес. Настройки относятся только к этому репозиторию.

Команда настройки часто завершается без текста — это нормально. Не вводите --global в рамках этой инструкции. Если появилось Author identity unknown, проверьте обе настройки именно в этой папке и повторите только неудавшийся коммит.

Источники: <https://git-scm.com/docs/git-init> и <https://git-scm.com/docs/git-config>

А3 Первый коммит

Редактирование в Блокноте

Откройте git-practice в Проводнике. Дважды щёлкните README.txt или выберите «Открыть с помощью», «Блокнот». В строке Student: YOUR_NAME замените YOUR_NAME своим именем. Остальной текст оставьте без изменений. Нажмите Ctrl+S и закройте Блокнот.

Вернитесь в то же окно Git Bash. Имя файла должно быть ровно README.txt.

Подготовка и сохранение истории

```
git add README.txt  
git status
```

Проверка. README.txt находится в Changes to be committed. Если add ничего не напечатал, это нормально.

```
git commit -m "Add project description"  
git status
```

Проверка. Ожидаем nothing to commit, working tree clean. Создан первый коммит. Его буквенно-цифровой идентификатор у каждого свой.

Если файл не найден

Выполните pwd и ls. Проверьте папку, имя и сохранение в Блокноте. Не создавайте файл заново и не повторяйте git init в другом месте. При неожиданном состоянии покажите преподавателю вывод status.

Источники: <https://git-scm.com/docs/git-add> и <https://git-scm.com/docs/git-commit>

A4 A6 Второй коммит и точка A

A4 Изменение и сравнение

В Блокноте откройте README.txt. Замените Status: draft на Status: in progress. Добавьте строку Note: и собственную короткую фразу, например Note: My first Git practice. Сохраните Ctrl+S и закройте Блокнот.

```
git status  
git --no-pager diff
```

Проверка. README.txt отмечен как modified. В сравнении видны старый статус со знаком минус, новый со знаком плюс и добавленная заметка. Знаки сравнения в сам файл не вписываем.

A5 Сохранение второго изменения

```
git add README.txt  
git commit -m "Update project status"
```

A6 Проверка точки A

```
git --no-pager log --oneline  
git status
```

Проверка. Два коммита, последняя запись сверху. Текущая ветка main. Состояние working tree clean. В файле ваше имя, новый статус и заметка.

Заполните первые ответы в report_template.txt: чем отличаются Ctrl+S, add и commit. Сохраните шаблон. Если закончили раньше, проверьте смысл каждой команды. Вторую часть начинайте после объяснения преподавателя.

Если не успели, запишите последний успешный шаг. На второй паре сначала завершите точку A. Во время перерыва сохраняйте папку. Для продолжения снова откройте Git Bash в той же git-practice.

Источник сравнения: <https://git-scm.com/docs/git-diff>

Б1 Б2 Ветка и план работы

Начинайте после точки А. Все команды по-прежнему вводятся в Git Bash внутри git-practice.

Б1 Создание ветки

```
git status
git switch -c feature-plan
git branch --show-current
```

Проверка. До переключения состояние чистое. После переключения имя ветки feature-plan.

Б2 Новый файл и коммит

Откройте Блокнот через «Пуск», создайте новый документ и напишите две строки плана. Например:

```
Plan: Improve the student guide
Step: Add a usage example
```

Это содержимое файла, его вводим в Блокноте. Через «Сохранить как» выберите git-practice и имя PLAN.txt. При необходимости тип «Все файлы», кодировка UTF-8. Закройте Блокнот. В Git Bash проверьте имя через ls.

```
ls
git add PLAN.txt
git commit -m "Add work plan"
git status
```

Проверка. В git-practice есть README.txt и PLAN.txt. Коммит сохранён на feature-plan, состояние чистое.

Если ветка уже существует, проверьте текущее имя. Переход на существующую ветку: git switch feature-plan. Если файл назван PLAN.txt.txt, включите показ расширений в Проводнике и исправьте имя.

Источник: <https://git-scm.com/docs/git-switch>

Б3 Б4 Объединение и метка версии

Б3 Возвращение в main и включение плана

```
git switch main
git branch --show-current
ls
```

Проверка. Текущая ветка main. PLAN.txt пока отсутствует: коммит с ним сохранён в другой ветке. Вручную ничего не удаляйте.

```
git merge --ff-only feature-plan
ls
git --no-pager log --oneline --all
```

Проверка. Ожидаем Fast-forward, оба файла и три коммита. Отдельный коммит слияния в этом маршруте не создаётся.

Б4 Метка учебной версии

```
git status
git tag v0.1.0
git tag --list
git --no-pager log --oneline --decorate
```

Проверка. Рабочая папка чистая. Метка v0.1.0 относится к последнему коммиту main. Ветка feature-plan сохраняется.

Если fast-forward невозможен, история отличается от учебной: покажите преподавателю `log --oneline --all`. Не применяйте команды удаления истории. Если метка уже есть, проверьте её положение, не пересоздавая автоматически.

Источники: <https://git-scm.com/docs/git-merge> и <https://git-scm.com/docs/git-tag>

Б5 Б6 Bundle и текстовая история

Б5 Создание и проверка bundle

Продолжайте из git-practice. ../submission означает соседнюю папку внутри git-start. Все необходимые изменения к этому моменту должны быть закоммичены.

```
git status
mkdir -p ../submission
git bundle create ../submission/work.bundle --all
git bundle verify ../submission/work.bundle
```

Проверка. Состояние чистое. В выводе проверки присутствует is okay. Bundle содержит сохранённую историю, ветки и метку, но не несохранённые правки или отдельный отчёт.

Б6 Текстовая история

Следующая строка — одна команда. Знак > записывает результат в файл, поэтому отсутствие списка коммитов в самом терминале нормально.

```
git log --oneline --decorate --all > ../submission/history.txt
ls ../submission
```

Проверка. В submission есть work.bundle и history.txt. Откройте history.txt в Блокноте: ожидаются три коммита, обозначения веток и метка v0.1.0.

Если после создания bundle вы сделали новые коммиты, повторите его создание, verify и запись history.txt. Не переименовывайте обычный текстовый файл в .bundle. Созданный Git файл должен оставаться двоичным.

Источник: <https://git-scm.com/docs/git-bundle>

Б7 Б8 Отчёт и ZIP для поздней сдачи

Б7 Короткий отчёт

Откройте `report_template.txt` из `git-start` в Блокноте. Укажите имя, состояние работы и короткие ответы своими словами. Через «Сохранить как» сохраните заполненный текст в `submission` под именем `report.txt`.

Ответьте: чем отличаются `add` и `commit`; что изменилось во втором коммите; почему `PLAN.txt` исчезал при переходе в `main`; для чего нужен `bundle`. Если возникла ошибка, приведите команду и её ответ. Большой отчёт со скриншотами не требуется.

Б8 Проверка и упаковка

```
git status
ls ../submission
```

Проверка. Точка Б: три коммита, два файла в `main`, метка `v0.1.0`, чистая рабочая папка. В `submission` ровно нужные `work.bundle`, `history.txt` и `report.txt`. `Verify` завершился успешно.

В Проводнике перейдите в `git-start`. Нажмите правой кнопкой по `submission`. Windows 11: сжатие в ZIP; Windows 10: «Отправить», «Сжатая ZIP-папка». Если нужного пункта нет, откройте «Показать дополнительные параметры». Назовите архив своей фамилией, например `Ivanov_Git.zip`. Допустимо, что внутри ZIP файлы находятся в папке `submission`.

Откройте ZIP в Проводнике и убедитесь, что видны три файла результата. Сохраните ZIP и рабочую папку до окончания проверки. При работе на общем компьютере перенесите свою копию в доступное вам постоянное место.

Когда преподаватель откроет задание в ЭИОС

Перейдите по сообщённой ссылке, прикрепите один ZIP, сохраните изменения и проверьте статус. Если есть отдельная кнопка отправки на оценку, нажмите её и подтвердите действие. Срок преподаватель сообщит отдельно. До этого загружать работу не требуется.

Критерии и типовые затруднения

Оценивание основной работы из 10 баллов

Критерий	Баллы
README.txt содержит имя студента и две осмысленные версии в истории	2
PLAN.txt добавлен отдельным коммитом в feature-plan и включён в main	3
Метка v0.1.0 у нужного коммита и чистое итоговое состояние	1
Bundle восстанавливается, history.txt согласуется с ним	2
report.txt содержит имя и четыре понятных ответа своими словами	2

Скорость выполнения и дополнительный конфликт не влияют на максимальный балл основной работы. Комментарии к ошибкам помогают преподавателю оценить достигнутый результат.

Что проверить перед вопросом

not a git repository: проверьте pwd, папку git-practice и выполнение A2.
pathspec ... did not match: проверьте ls, имя файла, его расширение и сохранение в Блокноте.

nothing to commit: прочитайте историю и status. Возможно, шаг уже выполнен или текст не изменён. Author identity unknown: вернитесь к настройкам автора A2.

Пустой diff: сохраните Блокнот и прочитайте status. Если уже выполнили add, используйте `git --no-pager diff --cached`. Предупреждение LF/CRLF само по себе не означает провал: проверьте, создан ли коммит, и итоговый status.

Ветка уже существует: для перехода используйте switch без -с. Неожиданные изменения или отказ fast-forward: покажите преподавателю `git status` и `git --no-pager log --oneline --all`.

Проверка. Вопрос формулируем так: «Шаг Б3. Ввёл команду ... Получил ответ ...». Не удаляем .git и не применяем `reset --hard` или `clean` ради попытки начать заново.

Дополнительно Конфликт в отдельной папке

Этот маршрут необязателен и не нужен для 10 баллов. Выполняйте его после сохранения основного ZIP. Основной git-practice и bundle не меняйте. Начальное место команд — Git Bash внутри git-practice.

Новая учебная история

Сначала убедитесь, что папки conflict-demo рядом ещё нет. Если она уже есть от прежней попытки, остановитесь и проверьте её состояние с преподавателем. Создание без -p намеренно сообщает о повторном запуске.

```
cd ..
mkdir conflict-demo
cd conflict-demo
git init -b main
git config user.name "Ivan Ivanov"
git config user.email "student@example.invalid"
echo "color=blue" > settings.txt
git add settings.txt
git commit -m "Add initial color"
```

Разные правки одной строки

```
git switch -c change-color
echo "color=green" > settings.txt
git add settings.txt
git commit -m "Choose green"
git switch main
echo "color=red" > settings.txt
git add settings.txt
git commit -m "Choose red"
```

Проверка. В этой дополнительной папке main и change-color содержат разные варианты одной строки. Перейдите к следующей странице. Команды echo здесь вводятся в Git Bash и записывают текст в файл.

Дополнительно Разрешение конфликта

Попытка объединения

```
git merge change-color
git status
```

Проверка. Ожидаем CONFLICT и both modified: settings.txt. Это запланированная ситуация дополнительного упражнения.

Откройте settings.txt в Блокноте. Git поместил внутрь обе версии и разделители. Верхний вариант относится к текущей main, нижний — к change-color. Решение о правильном содержимом принимает человек.

Исправление файла

Замените всё содержимое settings.txt одной строкой color=green. Удалите маркеры <<<<<<, ===== и >>>>>> вместе с лишними вариантами текста. Сохраните Ctrl+S, закройте Блокнот и вернитесь в Git Bash.

```
git add settings.txt
git commit -m "Resolve color conflict"
git status
git --no-pager log --oneline --graph --all
```

Проверка. Ожидаем working tree clean и коммит разрешения конфликта. В файле осталась ровно выбранная строка без маркеров.

Если хотите отложить дополнительное упражнение сразу после сообщения CONFLICT, до ручного редактирования можно выполнить git merge --abort. Для основной сдачи это упражнение не требуется. Если вы уже внесли правки и сомневаетесь, сначала уточните состояние у преподавателя.

Возвращение к основному проекту

```
cd ../git-practice
git status
```

Дополнительный conflict-demo хранится отдельно. Основной ZIP содержит только результат git-practice. По желанию можно кратко описать наблюдение о конфликте в отчёте и заново собрать ZIP с обновлённым report.txt.

Источник: <https://git-scm.com/docs/git-merge>